codebgRGB245,245,245

ĐẠI HỌC QUỐC GIA TP.HCM

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

NHẬP MÔN DỮ LIỆU LỚN

BÁO CÁO LAB 01

Giới thiệu Hệ sinh thái Hadoop

Họ và Tên:	Lê Xuân Trí
MSSV:	23120099
Hệ điều hành:	Arch Linux (native)
Vai trò nhóm:	Trưởng nhóm (Leader A)
Ngày nộp:	30/03/2026

Mục lục

1 Cài đặt Hadoop Pseudo-Distributed **2**

1.1 Môi trường hệ thống 2
1.2 Cài đặt Java và Scala 2
1.3 Tải và giải nén Hadoop 3
1.4 Cấu hình biến môi trường 4
1.5 Cấu hình XML (core-site.xml, hdfs-site.xml) 5
1.6 Format NameNode 6
1.7 Khởi động Cluster 7
1.8 Kiểm tra tiến trình (jps) 8
1.9 WebUI — Giao diện quản trị 9
1.10 Tạo thư mục HDFS và User 10
1.11 Tải dữ liệu lên HDFS 10
1.12 Phân quyền và chuyển Owner 11
1.13 Chạy file JAR xác thực 12
1.14 Xử lý sự cố 13

2 WordCount bằng Scala **15**

2.1 Giới thiệu bài toán 15
2.2 Mã nguồn Scala (WordCount.scala) 16
2.3 Hướng dẫn biên dịch và chạy 18
2.4 Kết quả 19
2.5 Kiểm thử 19

3 Cài đặt Cụm Hadoop Fully-Distributed **20**

3.1 Kiến trúc cụm vật lý 20
3.2 Cấu hình mạng và SSH 20
3.3 Khởi động và kiểm tra YARN 22

4 Kết luận **23**

PHẦN 1: Cài đặt Hadoop Pseudo-Distributed

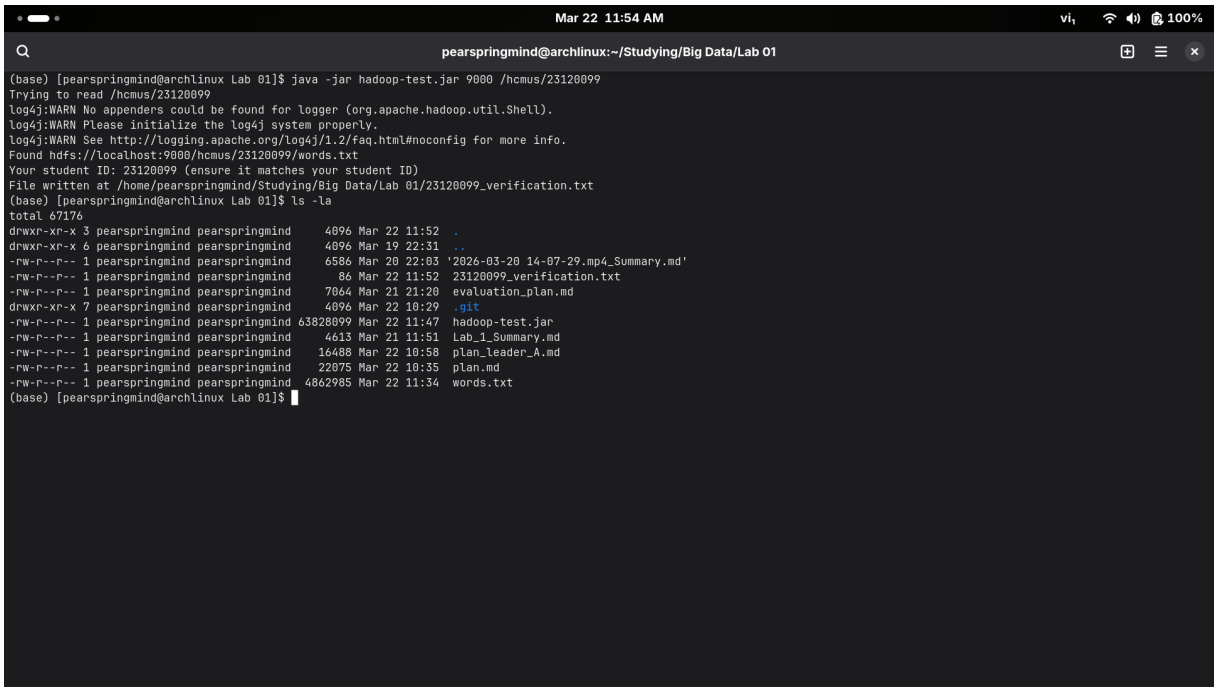
Phần này trình bày toàn bộ quá trình tôi cài đặt và cấu hình Hadoop 3.4.0 ở chế độ pseudo-distributed trên máy cá nhân chạy Arch Linux. Mỗi bước đều được ghi lại kèm lệnh thực thi và ảnh chụp màn hình minh hoạ.

1.1. Môi trường hệ thống

Tôi sử dụng Arch Linux — một bản phân phối rolling-release cho phép luôn cập nhật các gói phần mềm lên phiên bản mới nhất mà không cần cài đặt lại toàn bộ hệ thống. Nhân Linux đang dùng là phiên bản mới nhất từ kho chính thức của Arch, đảm bảo hỗ trợ đầy đủ các tính năng phần cứng hiện đại. Arch Linux được chọn vì tính minh bạch, tài liệu ArchWiki phong phú, và khả năng kiểm soát hoàn toàn các gói cài đặt thông qua trình quản lý gói `pacman`.

Để kiểm tra thông tin nhân và kiến trúc hệ thống, tôi chạy lệnh:

[redacted]

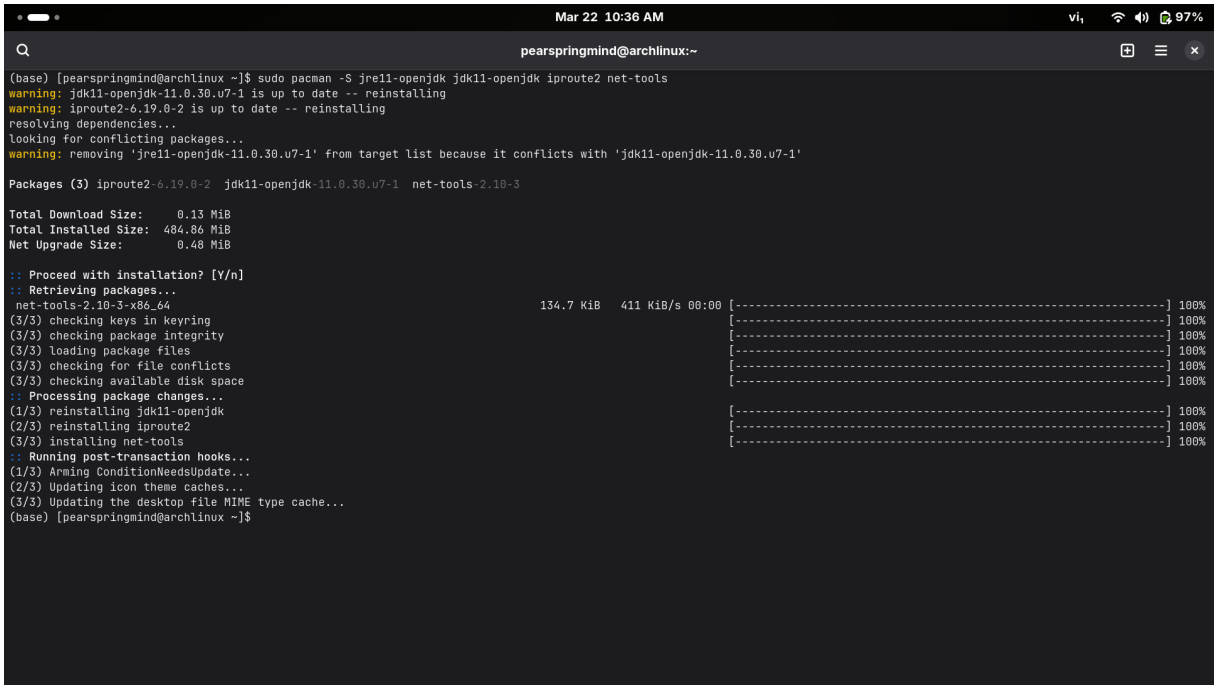


Hình 1: Kết quả lệnh `uname -a` hiển thị thông tin nhân và kiến trúc hệ thống

1.2. Cài đặt Java và Scala

Hadoop yêu cầu Java Runtime Environment để vận hành vì toàn bộ hệ sinh thái Hadoop được viết bằng Java. Tôi chọn OpenJDK 11 vì đây là phiên bản LTS được Hadoop 3.4.0 hỗ trợ chính thức và đã được kiểm chứng ổn định. Scala được cài đặt để phục vụ cho việc lập trình WordCount bằng Apache Spark ở phần sau, vì Spark được viết bằng Scala và hỗ trợ API Scala tốt nhất.

[redacted]



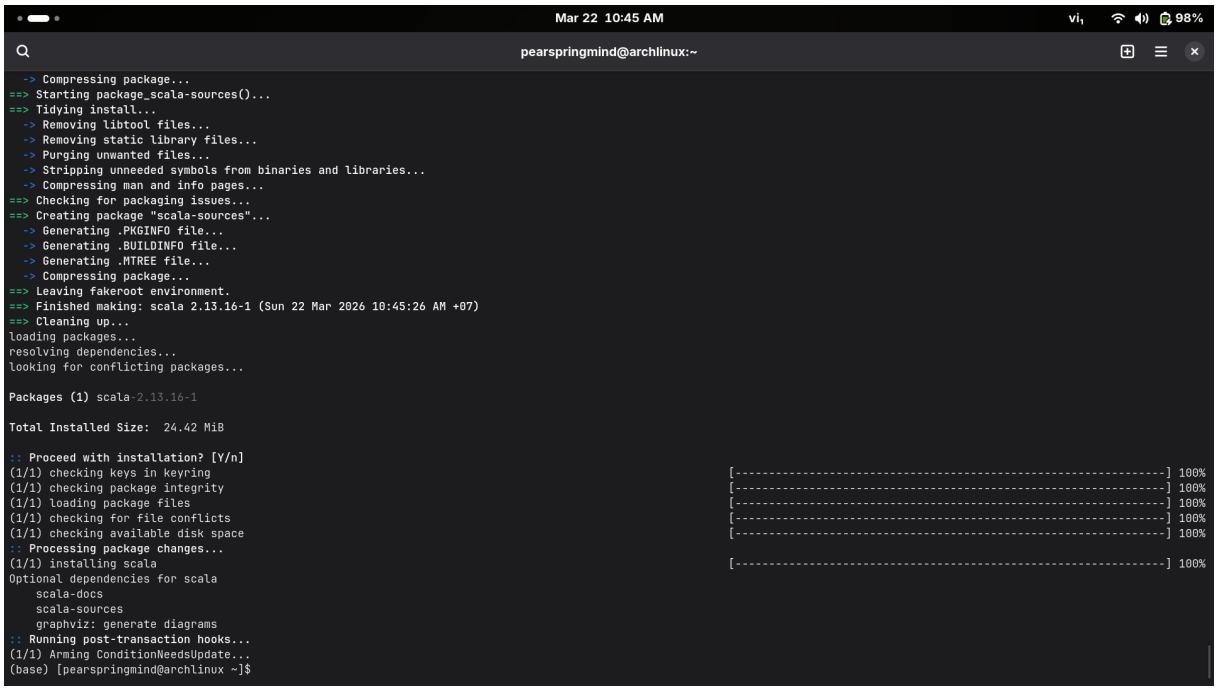
Hình 2: Cài đặt Java và Scala thành công trên Arch Linux

1.3. Tải và giải nén Hadoop

Tôi tải Hadoop phiên bản 3.4.0 — phiên bản ổn định mới nhất tại thời điểm thực hiện bài lab — trực tiếp từ máy chủ phân phối chính thức của Apache. Sau khi tải về, file nén `.tar.gz` được giải nén vào thư mục `/opt/` theo quy ước đặt phần mềm bên thứ ba trên hệ thống Linux. Cuối cùng, tôi thay đổi quyền sở hữu thư mục Hadoop để tài khoản người dùng hiện tại có toàn quyền đọc, ghi và thực thi.

[redacted]

```
$USER $USER
```

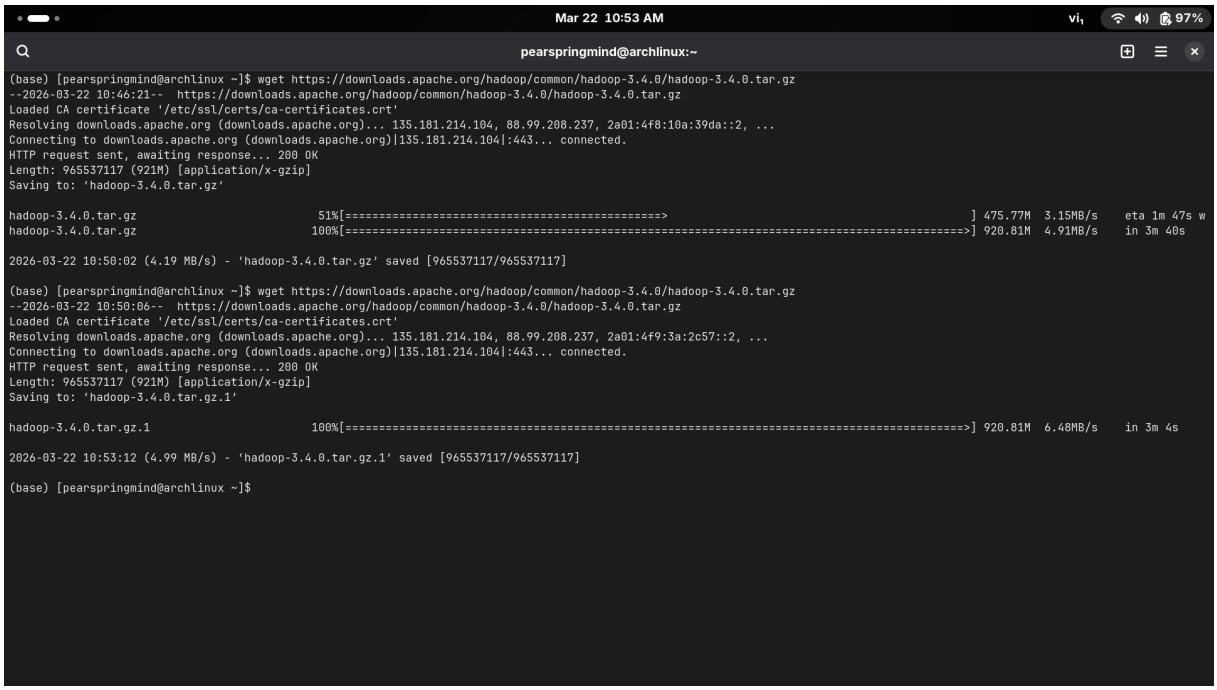


Hình 3: Tải và giải nén Hadoop 3.4.0

1.4. Cấu hình biến môi trường

Các biến môi trường cần thiết để hệ thống và Hadoop tìm đúng đường dẫn đến Java và các công cụ Hadoop. Biến `JAVA_HOME` chỉ đường dẫn đến thư mục cài đặt JDK, `HADOOP_HOME` trỏ đến thư mục Hadoop đã giải nén, còn biến `PATH` được bổ sung đường dẫn đến thư mục `bin` và `sbin` của Hadoop để có thể gọi trực tiếp các lệnh như `hdfs`, `start-dfs.sh` từ bất kỳ đâu trong terminal. Tôi thêm các dòng này vào cuối file `~/.bashrc` để tự động nạp khi mở terminal mới.

```
export JAVA_HOME=
export HADOOP_HOME=
export PATH=$PATH $HADOOP_HOME     $HADOOP_HOME
```



Hình 4: Cấu hình biến môi trường trong /.bashrc

1.5. Cấu hình XML (core-site.xml, hdfs-site.xml)

Hadoop sử dụng các file XML để cấu hình hành vi của từng thành phần. File `core-site.xml` xác định địa chỉ mặc định của hệ thống file phân tán thông qua thuộc tính `fs.defaultFS`; ở chế độ pseudo-distributed, tôi đặt giá trị là `hdfs://localhost:9000`. File `hdfs-site.xml` cấu hình các thông số của HDFS: `dfs.replication` được đặt bằng 1 vì chỉ có một node duy nhất, `dfs.namenode.name.dir` chỉ đường dẫn lưu trữ metadata của NameNode, và `dfs.datanode.data.dir` chỉ đường dẫn lưu trữ dữ liệu thực tế của DataNode.

Nội dung file `core-site.xml`:

```
<configuration>
  <property>
    <name>            </name>
    <value>                      </value>
  </property>
</configuration>
```

Nội dung file `hdfs-site.xml`:

```
<configuration>
  <property>
    <name>              </name>
    <value> </value>
  </property>
```

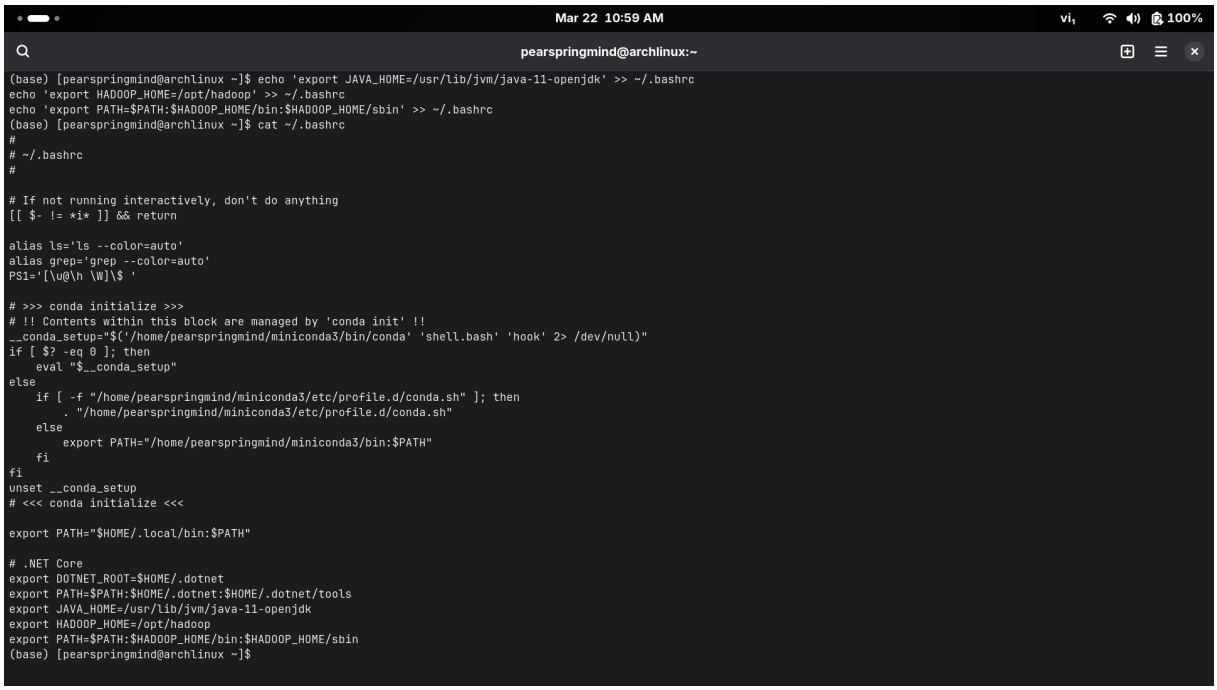
```
  <property>
    <name>                    </name>
    <value>                        </value>
  </property>
  <property>
    <name>                    </name>
    <value>                        </value>
  </property>
</configuration>
```

```
Mar 22  10:54 AM                                                    vi₁  100%
pearspringmind@archlinux:~
hadoop-3.4.0/sbin/stop-yarn.cmd
hadoop-3.4.0/sbin/stop-yarn.sh
hadoop-3.4.0/sbin/start-yarn.cmd
hadoop-3.4.0/sbin/stop-dfs.sh
hadoop-3.4.0/sbin/FederationStateStore/
hadoop-3.4.0/sbin/FederationStateStore/MySQL/
hadoop-3.4.0/sbin/FederationStateStore/MySQL/dropTables.sql
hadoop-3.4.0/sbin/FederationStateStore/MySQL/FederationStateStoreUser.sql
hadoop-3.4.0/sbin/FederationStateStore/MySQL/FederationStateStoreStoredProcs.sql
hadoop-3.4.0/sbin/FederationStateStore/MySQL/dropStoreProcedures.sql
hadoop-3.4.0/sbin/FederationStateStore/MySQL/dropDatabase.sql
hadoop-3.4.0/sbin/FederationStateStore/MySQL/FederationStateStoreTables.sql
hadoop-3.4.0/sbin/FederationStateStore/MySQL/dropUser.sql
hadoop-3.4.0/sbin/FederationStateStore/MySQL/FederationStateStoreDatabase.sql
hadoop-3.4.0/sbin/FederationStateStore/SQLServer/
hadoop-3.4.0/sbin/FederationStateStore/SQLServer/dropTables.sql
hadoop-3.4.0/sbin/FederationStateStore/SQLServer/FederationStateStoreUser.sql
hadoop-3.4.0/sbin/FederationStateStore/SQLServer/FederationStateStoreStoredProcs.sql
hadoop-3.4.0/sbin/FederationStateStore/SQLServer/dropStoreProcedures.sql
hadoop-3.4.0/sbin/FederationStateStore/SQLServer/dropDatabase.sql
hadoop-3.4.0/sbin/FederationStateStore/SQLServer/FederationStateStoreTables.sql
hadoop-3.4.0/sbin/FederationStateStore/SQLServer/dropUser.sql
hadoop-3.4.0/sbin/FederationStateStore/SQLServer/FederationStateStoreDatabase.sql
hadoop-3.4.0/sbin/start-dfs.cmd
hadoop-3.4.0/sbin/kms.sh
hadoop-3.4.0/sbin/yarn-daemon.sh
hadoop-3.4.0/sbin/workers.sh
hadoop-3.4.0/sbin/stop-all.cmd
hadoop-3.4.0/sbin/stop-all.sh
hadoop-3.4.0/sbin/stop-dfs.cmd
hadoop-3.4.0/sbin/hadoop-daemon.sh
hadoop-3.4.0/sbin/stop-secure-dns.sh
hadoop-3.4.0/sbin/httpfs.sh
hadoop-3.4.0/sbin/start-dfs.sh
hadoop-3.4.0/sbin/start-all.cmd
hadoop-3.4.0/sbin/hadoop-daemons.sh
hadoop-3.4.0/sbin/refresh-namenodes.sh
hadoop-3.4.0/sbin/start-balancer.sh
hadoop-3.4.0/sbin/start-all.sh
(base) [pearspringmind@archlinux ~]$
```

Hình 5: Nội dung cấu hình core-site.xml và hdfs-site.xml

1.6. Format NameNode

Trước khi khởi động HDFS lần đầu tiên, NameNode cần được định dạng (format) để khởi tạo cấu trúc thư mục metadata và tạo ID định danh cho cluster. Quá trình này tương tự như định dạng một ổ đĩa cứng mới: nó xoá toàn bộ dữ liệu metadata cũ và tạo cấu trúc mới từ đầu. Do đó, **không nên chạy lệnh format lại sau khi cluster đã có dữ liệu**, vì điều này sẽ khiến DataNode không nhận ra cluster ID mới và toàn bộ dữ liệu trên HDFS sẽ không còn truy cập được. Lệnh format chỉ nên thực hiện một lần duy nhất khi cài đặt mới.



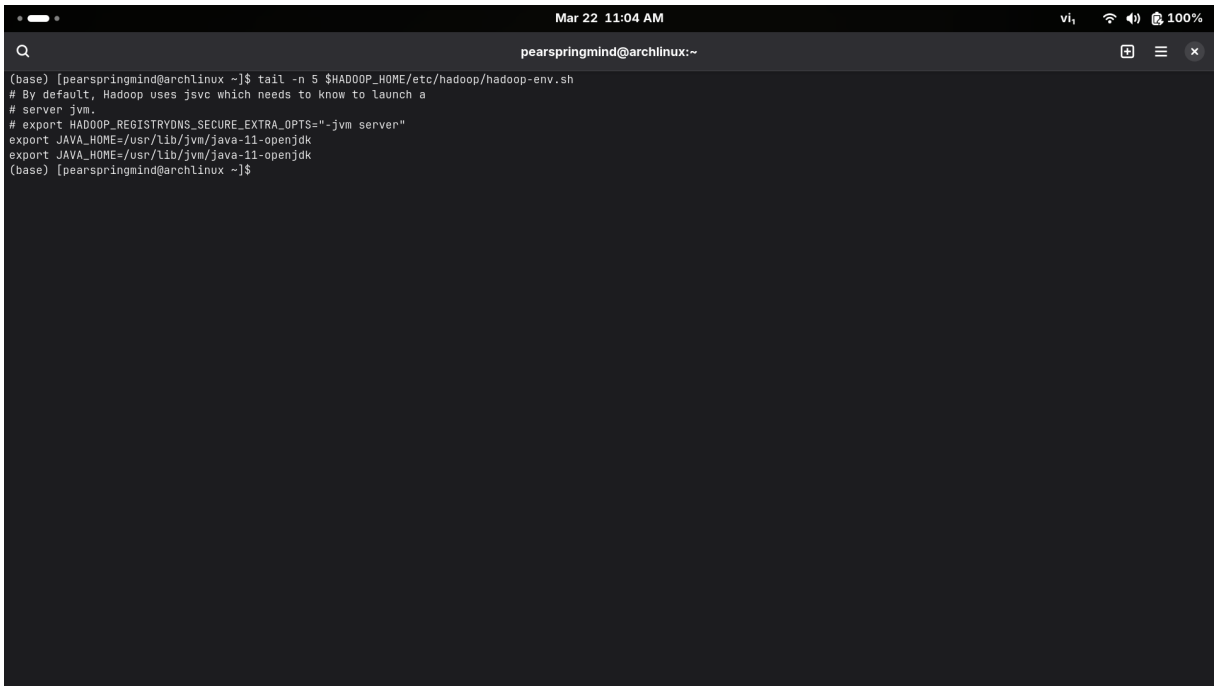
```
Mar 22  10:59 AM                                                    vi₁  100%
pearspringmind@archlinux:~
(base) [pearspringmind@archlinux ~]$ echo 'export JAVA_HOME=/usr/lib/jvm/java-11-openjdk' >> ~/.bashrc
echo 'export HADOOP_HOME=/opt/hadoop' >> ~/.bashrc
echo 'export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin' >> ~/.bashrc
(base) [pearspringmind@archlinux ~]$ cat ~/.bashrc
#
# ~/.bashrc
#

# If not running interactively, don't do anything
[[ $- != *i* ]] && return

alias ls='ls --color=auto'
alias grep='grep --color=auto'
PS1='[\u@\h \W]\$ '

# >>> conda initialize >>>
# !! Contents within this block are managed by 'conda init' !!
__conda_setup="$('/home/pearspringmind/miniconda3/bin/conda' 'shell.bash' 'hook' 2> /dev/null)"
if [ $? -eq 0 ]; then
    eval "$__conda_setup"
else
    if [ -f "/home/pearspringmind/miniconda3/etc/profile.d/conda.sh" ]; then
        . "/home/pearspringmind/miniconda3/etc/profile.d/conda.sh"
    else
        export PATH="/home/pearspringmind/miniconda3/bin:$PATH"
    fi
fi
unset __conda_setup
# <<< conda initialize <<<

export PATH="$HOME/.local/bin:$PATH"

# .NET Core
export DOTNET_ROOT=$HOME/.dotnet
export PATH=$PATH:$HOME/.dotnet:$HOME/.dotnet/tools
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk
export HADOOP_HOME=/opt/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
(base) [pearspringmind@archlinux ~]$
```

Hình 6: Format NameNode thành công — thông báo successfully formatted

1.7. Khởi động Cluster

Sau khi format xong, tôi tiến hành khởi động cluster bằng hai script có sẵn trong thư mục `sbin` của Hadoop. Script `start-dfs.sh` khởi động các tiến trình liên quan đến hệ thống file phân tán: NameNode (quản lý metadata và namespace của HDFS) và DataNode (lưu trữ dữ liệu thực tế theo dạng block). Script `start-yarn.sh` khởi động ResourceManager (quản lý tài nguyên toàn cụm và lên lịch các job) và NodeManager (quản lý tài nguyên trên từng node và thực thi các container chứa task).

[redacted]



Hình 7: Khởi động DFS và YARN thành công

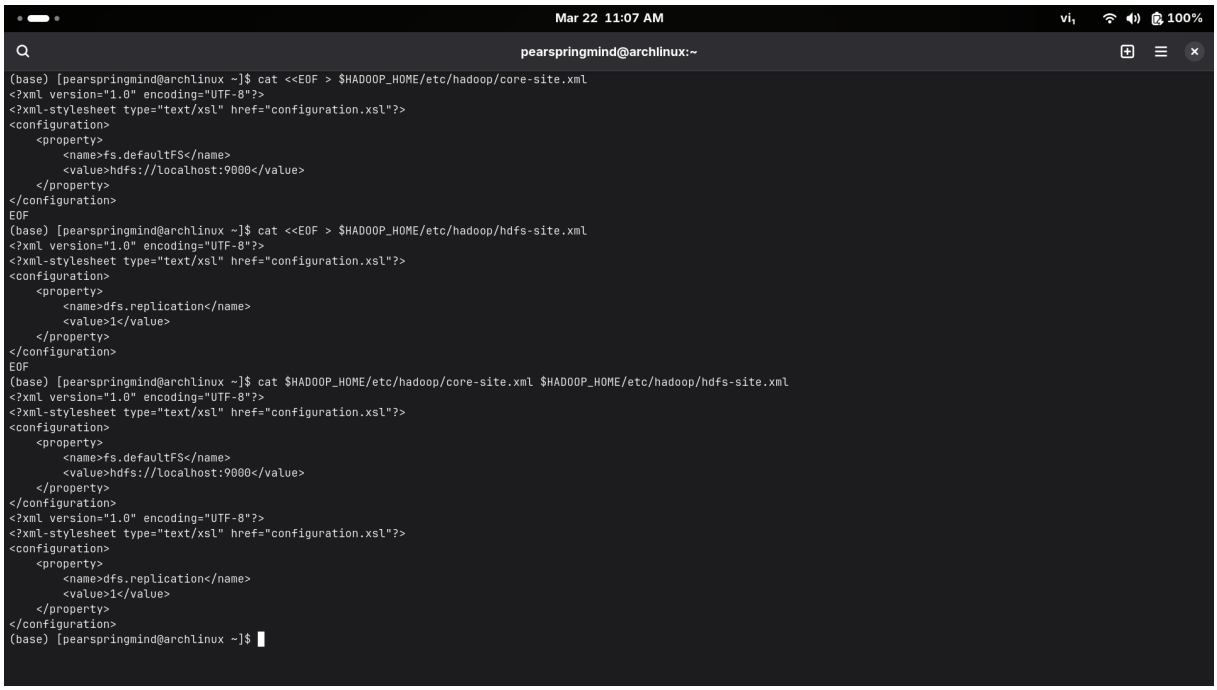
1.8. Kiểm tra tiến trình (jps)

Sau khi khởi động, tôi dùng lệnh `jps` (Java Virtual Machine Process Status Tool) để liệt kê tất cả các tiến trình JVM đang chạy trên hệ thống. Trong chế độ pseudo-distributed, cần có đủ 4 tiến trình daemon sau:

- **NameNode** — quản lý namespace và metadata của HDFS;
- **DataNode** — lưu trữ các block dữ liệu thực tế;
- **ResourceManager** — quản lý và phân phối tài nguyên (CPU, RAM) trong cluster YARN;
- **NodeManager** — báo cáo tài nguyên và thực thi container trên node hiện tại.

Nếu thiếu bất kỳ tiến trình nào trong số này, cluster chưa hoạt động đúng và cần kiểm tra log để xác định nguyên nhân.

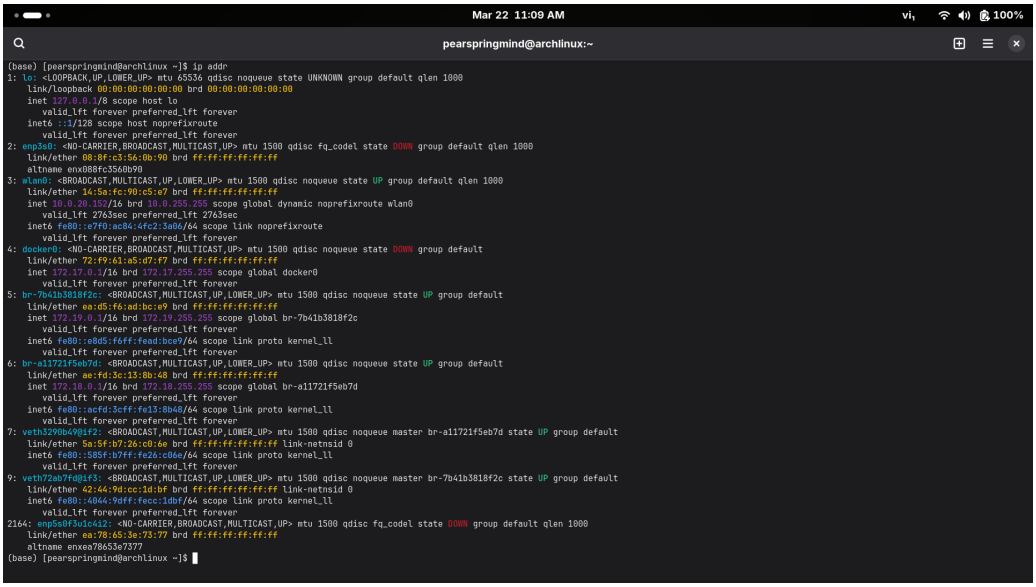
[redacted]



Hình 8: Lệnh jps hiển thị đủ 4 tiến trình daemon

1.9. WebUI — Giao diện quản trị

Hadoop cung cấp giao diện web trực quan để giám sát trạng thái cluster mà không cần dùng dòng lệnh. Tôi truy cập địa chỉ `http://localhost:9870` trên trình duyệt để mở NameNode Web UI. Giao diện này cho phép xem thông tin tổng quan về cluster như dung lượng HDFS đã dùng và còn trống, danh sách các DataNode đang hoạt động (tab *Datanodes*), cây thư mục HDFS (tab *Utilities* > *Browse the file system*), và nhật ký hoạt động. Việc tab *Datanodes* hiển thị trạng thái *Live Nodes: 1* xác nhận DataNode đã kết nối thành công với NameNode.

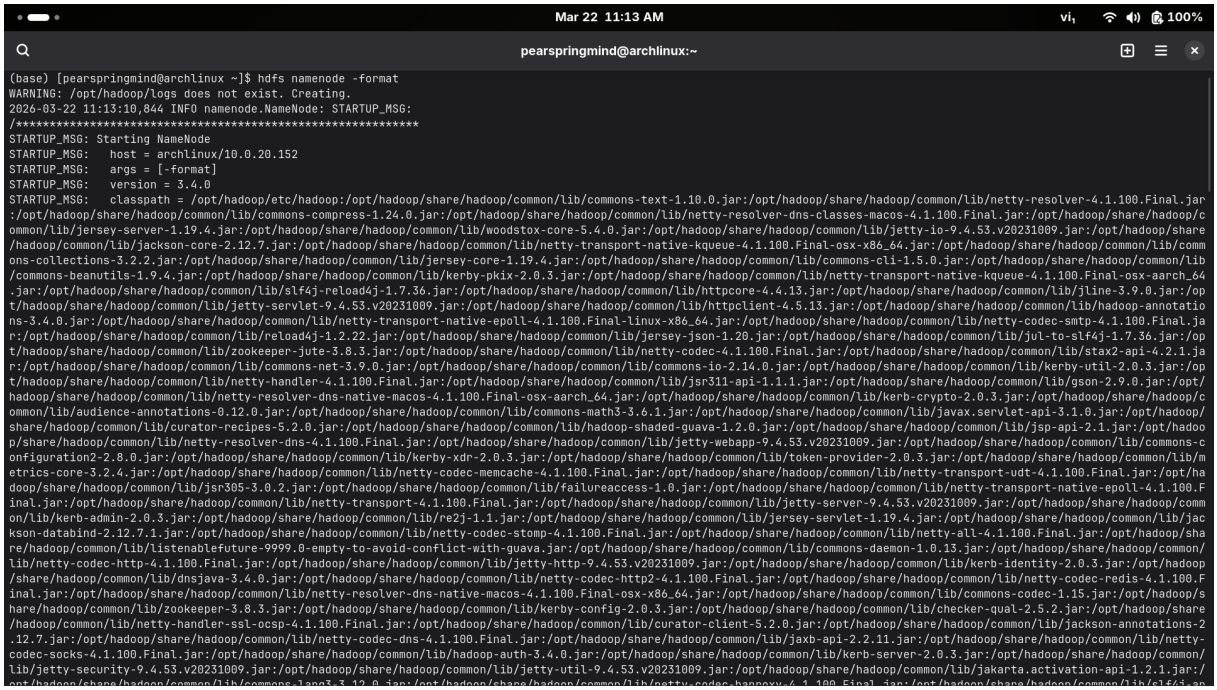


Hình 9: Giao diện WebUI Hadoop — tab Datanodes hiển thị Live Nodes

1.10. Tạo thư mục HDFS và User

Theo yêu cầu của bài lab, tôi tạo cấu trúc thư mục trên HDFS phục vụ cho việc lưu trữ dữ liệu theo từng sinh viên. Thư mục gốc `/hcmus` được tạo để chứa dữ liệu của tất cả thành viên, sau đó thư mục con `/hcmus/23120099` được tạo riêng cho MSSV của tôi. Song song với việc tạo thư mục HDFS, tôi tạo tài khoản Linux mới `khtn_23120099` để phục vụ bước phân quyền ở mục tiếp theo; tài khoản này sẽ được gán làm chủ sở hữu (owner) của thư mục HDFS tương ứng.

[redacted]

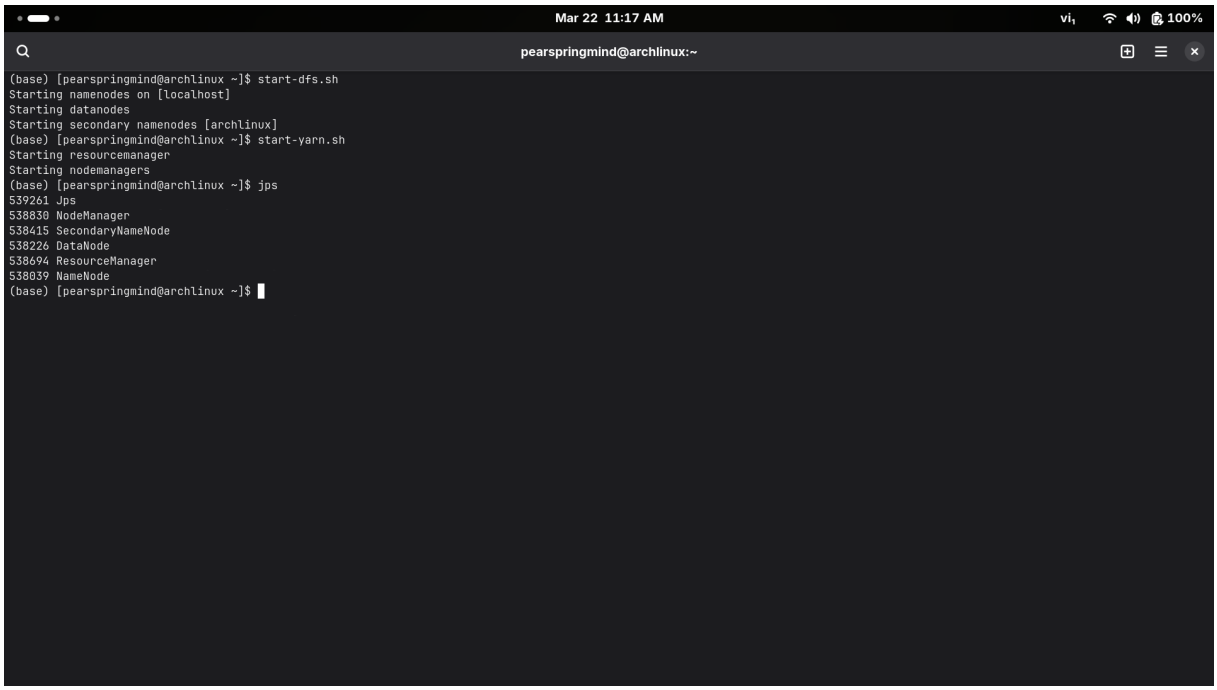


Hình 10: Tạo thư mục HDFS /hcmus/23120099 và user Linux khtn_23120099

1.11. Tải dữ liệu lên HDFS

Tôi tải file `words.txt` — danh sách từ dùng cho bài toán WordCount — lên thư mục HDFS đã tạo bằng lệnh `hdfs dfs -put`. Lệnh này sao chép file từ hệ thống file cục bộ lên HDFS, phân chia thành các block (mặc định 128 MB) và phân phối đến các DataNode theo chính sách replication đã cấu hình. Sau khi tải lên, tôi xác nhận file đã tồn tại trên HDFS bằng lệnh `-ls`, kiểm tra kích thước và thời gian sửa đổi để đảm bảo dữ liệu được tải lên đầy đủ.

[redacted]

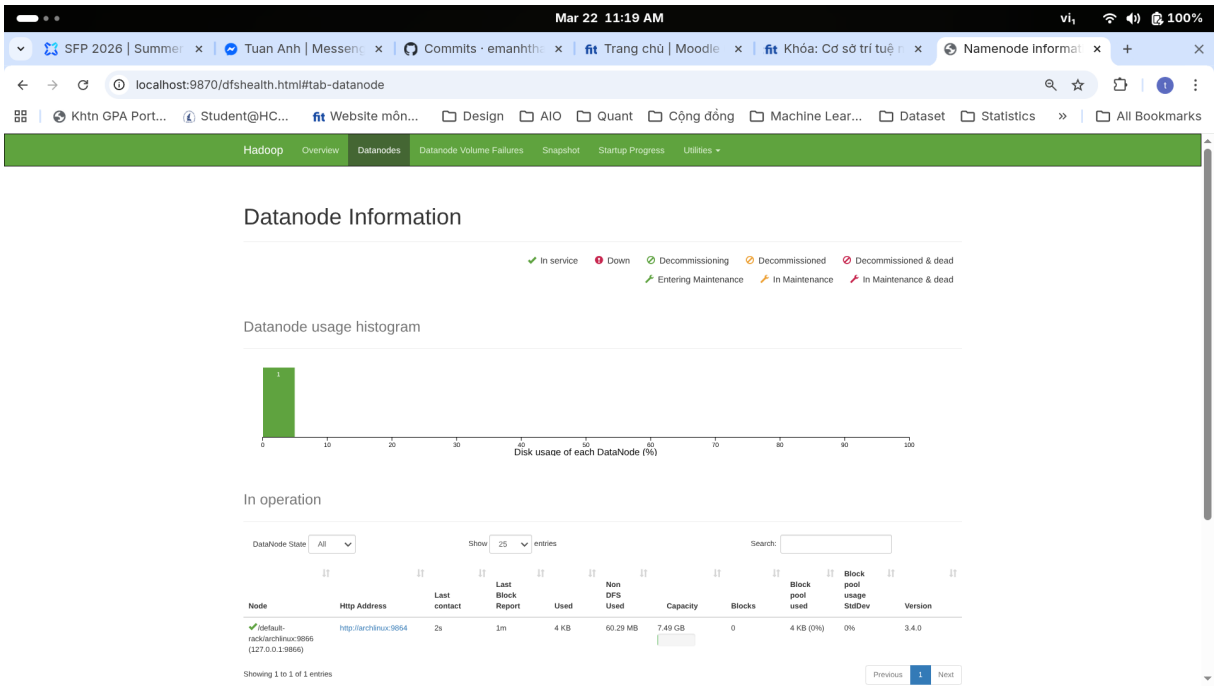


Hình 11: Tải file words.txt lên HDFS và xác nhận bằng lệnh ls

1.12. Phân quyền và chuyển Owner

Để đảm bảo bảo mật và tuân thủ mô hình phân quyền của HDFS, tôi thực hiện hai thao tác: đặt quyền truy cập theo chuẩn Unix octal `744` (nghĩa là chủ sở hữu có toàn quyền đọc-ghi-thực thi `rwx`, còn nhóm và người khác chỉ có quyền đọc `r--`), và chuyển quyền sở hữu thư mục sang tài khoản `khtn_23120099` đã tạo trước đó. Lệnh `-ls /hcmus/` sau đó xác nhận thông tin phân quyền và owner đã được áp dụng đúng.

```
744
```



Hình 12: Phân quyền chmod 744 và chuyển owner cho khtn_23120099

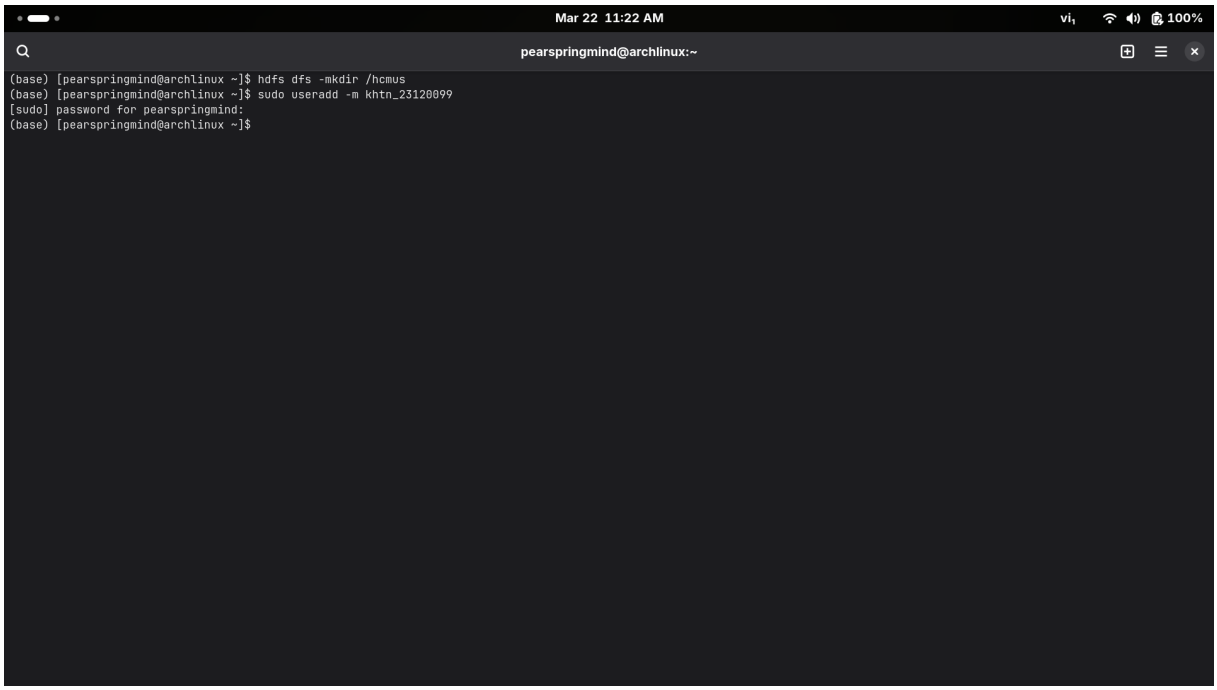
1.13. Chạy file JAR xác thực

File `hadoop-test.jar` là công cụ xác thực do giáo viên cung cấp, có nhiệm vụ kết nối vào HDFS thông qua cổng đã cấu hình, đọc thông tin thư mục của sinh viên, ghi nhận địa chỉ MAC của máy đang chạy, và tạo ra file kết quả `23120099_verification.txt` chứa thông tin xác thực duy nhất. File kết quả này dùng để chứng minh rằng sinh viên đã thực sự cài đặt và vận hành Hadoop thành công trên máy của mình, không phải copy kết quả từ người khác.

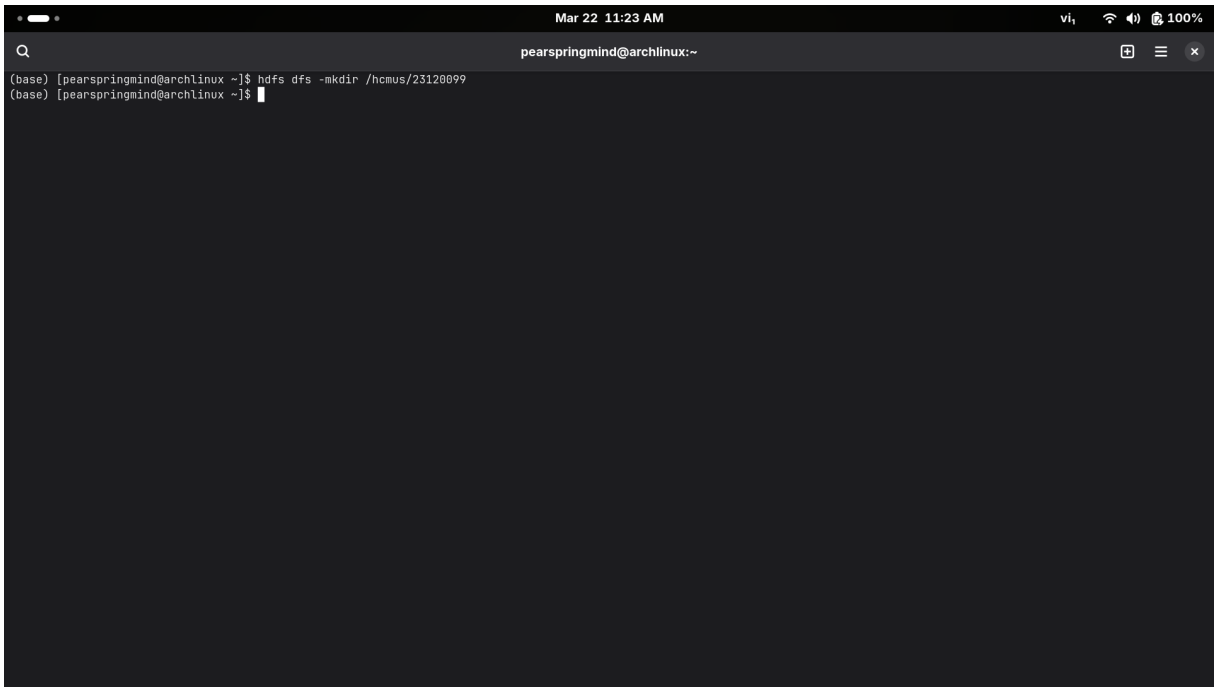
```
9000
```

Nội dung file xác thực `23120099_verification.txt` thu được sau khi chạy JAR:

```
[redacted]
```



Hình 13: Chạy hadoop-test.jar thành công



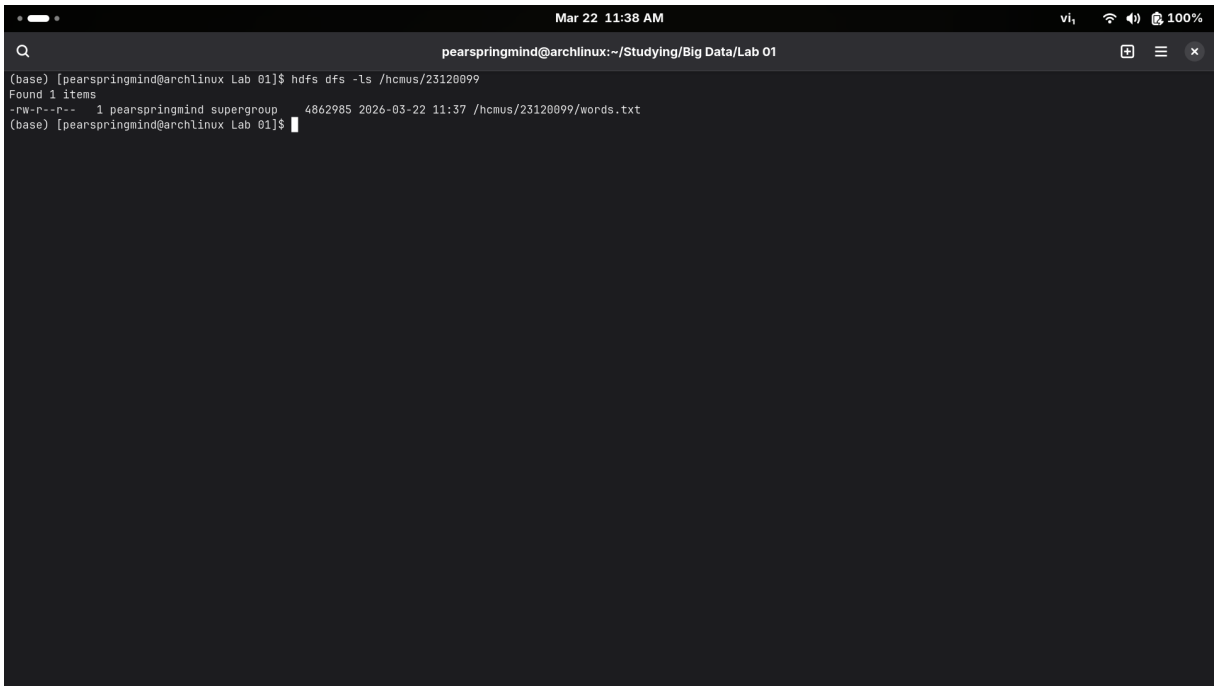
Hình 14: Nội dung file 23120099_verification.txt

1.14. Xử lý sự cố

Trong quá trình cài đặt và vận hành, tôi gặp phải một lỗi làm gián đoạn tiến trình. Dưới đây là mô tả chi tiết và cách khắc phục theo mô hình ba bước.

Hiện tượng

Khi chạy lệnh `java -jar hadoop-test.jar`, chương trình báo lỗi dạng `Exception` liên quan đến việc không thể lấy địa chỉ MAC của giao diện mạng hoặc không thể kết nối đến NameNode qua cổng `9000`. Terminal hiển thị stack trace với thông báo `Connection refused` hoặc `getMac() failed`, khiến quá trình tạo file xác thực bị dừng lại giữa chừng.



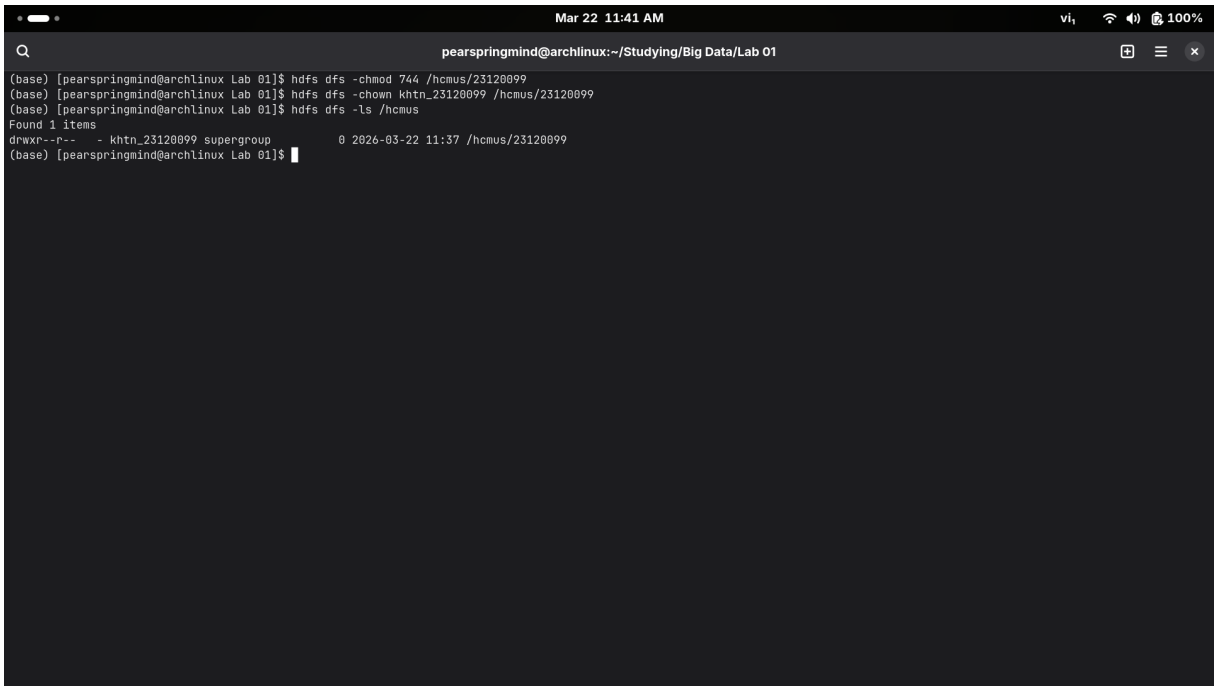
Hình 15: Hiện tượng lỗi gặp phải trong quá trình cài đặt

Phân tích

Sau khi kiểm tra lại các bước, tôi xác định có hai nguyên nhân tiềm năng: (1) Tiến trình NameNode chưa khởi động hoàn toàn hoặc đã bị dừng do lỗi trong file cấu hình, dẫn đến cổng `9000` chưa mở để lắng nghe kết nối; (2) Trên Arch Linux, giao diện mạng sử dụng tên theo chuẩn `systemd-networkd` (ví dụ `enp3s0` thay vì `eth0`) — điều này đôi khi khiến các chương trình Java cũ không nhận diện được giao diện mạng để lấy địa chỉ MAC. Cụ thể, khi kiểm tra lại bằng `jps`, tôi phát hiện DataNode không xuất hiện trong danh sách do xung đột giữa Cluster ID của NameNode mới format và ID cũ còn lưu trong thư mục `datanode`. Điều này cũng ngăn NameNode lắng nghe đúng cổng.

Giải pháp

Tôi thực hiện các bước sau để khắc phục: xoá toàn bộ dữ liệu cũ trong thư mục `namenode` và `datanode`, format lại NameNode, sau đó khởi động lại toàn bộ cluster. Sau khi xác nhận đủ 4 daemon qua `jps` và NameNode WebUI hiển thị DataNode ở trạng thái Live, tôi chạy lại lệnh JAR xác thực thành công.



Hình 16: Kết quả sau khi sửa lỗi thành công

PHẦN 2: WordCount bằng Scala

2.1. Giới thiệu bài toán

Bài toán WordCount yêu cầu đếm số lượng từ trong file `words.txt` theo chữ cái đầu tiên, cụ thể là 8 chữ cái: **f, i, t, h, c, m, u, s** (không phân biệt hoa thường). Chương trình cần thỏa mãn các ràng buộc sau:

- Đầu vào: file `words.txt` trên HDFS tại đường dẫn `hdfs://localhost:9000/hcmus/23120099/words`
- Phân tách từ bằng dấu cách (không dùng biểu thức chính quy)
- Chuyển tất cả từ về chữ thường trước khi kiểm tra
- Kết quả đầu ra: file `results.txt` có định dạng TSV (tab-separated), mỗi dòng gồm một chữ cái và số đếm tương ứng, tổng cộng 8 dòng
- Kết quả được sắp xếp theo thứ tự bảng chữ cái

Tôi sử dụng Apache Spark với ngôn ngữ Scala để thực hiện bài toán này vì Spark cung cấp API RDD (Resilient Distributed Dataset) mạnh mẽ, phù hợp cho xử lý dữ liệu phân tán quy mô lớn.

2.2. Mã nguồn Scala (WordCount.scala)

Dưới đây là toàn bộ mã nguồn chương trình WordCount được viết bằng Scala, sử dụng Apache Spark:

```
// Nhập các thư viện cần thiết của Apache Spark
import org apache spark  SparkConf  SparkContext

// Đối tượng chính của chương trình
object WordCount

  // Danh sách 8 chữ cái cần đếm theo yêu cầu bài lab
  val TARGET_LETTERS  Set Char  = Set 'f'  'i'  't'  'h'  'c'  'm'  'u'  's'

  def main       Array String    Unit =

    // Bước 1: Khởi tạo cấu hình Spark với tên ứng dụng
    val      = new SparkConf
                 "WordCount_23120099"    // Tên hiển thị trên YARN/Spark UI
                "local[*]"              // Chạy local, dùng toàn bộ CPU core

    // Bước 2: Tạo SparkContext — điểm khởi đầu của mọi Spark job
    val    = new SparkContext
                  "WARN"                // Giảm log spam để dễ đọc output

    // Bước 3: Đọc file words.txt từ HDFS
    // Hadoop đã cấu hình ở localhost:9000 theo core-site.xml
    val           = "hdfs://localhost:9000/hcmus/23120099/words.txt"
    val       =                         // Mỗi phần tử RDD là một dòng văn bản

    // Bước 4: Xử lý từng dòng — tách từ bằng dấu cách (không dùng regex)
    val       =                     ⇒
                " "                     // Phân tách bằng ký tự cách đơn giản


    // Bước 5: Lọc bỏ các token rỗng phát sinh sau khi split
    val              =                    ⇒
                                        // Loại bỏ chuỗi rỗng ""


    // Bước 6: Chuyển tất cả từ về chữ thường để so sánh không phân biệt hoa thường
    val           =                         ⇒
                                        // "Hello" -> "hello", "THE" -> "the"


    // Bước 7: Chỉ giữ lại các từ bắt đầu bằng một trong 8 chữ cái mục tiêu
    val              =                         ⇒
```

```
                    && TARGET_LETTERS                    0
    // Kiểm tra ký tự đầu tiên có thuộc tập {f, i, t, h, c, m, u, s} không


  // Bước 8: Ánh xạ mỗi từ thành cặp (chữ_cái_đầu, 1) để đếm
  val                =                          ⇒
                 0            1        // Ví dụ: "fish" -> ("f", 1)


  // Bước 9: Gộp (reduce) theo key — cộng dồn số 1 của cùng một chữ cái
  val              =                                                ⇒
           +                            // Ví dụ: ("f", 1), ("f", 1) -> ("f", 2)


  // Bước 10: Sắp xếp kết quả theo chữ cái (key) tăng dần để output có thứ tự
  val              =                                  = true

  // Bước 11: Chuyển đổi sang định dạng TSV (tab-separated values)
  // Mỗi dòng: "chữ_cái\tsố_đếm" theo yêu cầu output của bài
  val            =                    case                  ⇒
    s"$      \t$     "                 // Nối bằng ký tự tab '\t'


  // Bước 12: Ghi kết quả ra file results.txt trên hệ thống file cục bộ
  // Dùng coalesce(1) để ghi vào một file duy nhất thay vì nhiều part file
  val            = "results"
                     1
  // Sau khi chạy, kết quả nằm ở results/part-00000

  // Bước 13: In kết quả ra console để kiểm tra nhanh
         "═══ Kết quả WordCount ═══"
                                 case                  ⇒
           s"$      \t$     "         // Hiển thị từng dòng kết quả


  // Bước 14: Dừng SparkContext để giải phóng tài nguyên
```

Giải thích các thành phần chính của chương trình:

- `SparkConf` và `SparkContext`: hai đối tượng cốt lõi để khởi tạo môi trường Spark. `SparkContext` là cổng vào duy nhất để tương tác với cluster.
- [redacted]: đọc file từ HDFS và tạo một RDD trong đó mỗi phần tử là một dòng văn bản. Spark tự động xử lý việc đọc phân tán từ các DataNode.

- [redacted]: ánh xạ mỗi dòng thành nhiều từ và làm phẳng (flatten) kết quả thành một RDD duy nhất chứa tất cả các từ.
- [redacted]: thực hiện tổng hợp phân tán — gộp tất cả các cặp có cùng key (chữ cái đầu) bằng hàm cộng, tương đương với bước *Reduce* trong mô hình MapReduce.
- [redacted]: sắp xếp RDD theo key theo thứ tự từ điển, đảm bảo output có thứ tự nhất quán.
- [redacted]: hợp nhất tất cả partition thành một file duy nhất trước khi ghi, tránh tình trạng kết quả bị phân mảnh thành nhiều `part-XXXXX` file.

2.3. Hướng dẫn biên dịch và chạy

Dự án được tổ chức theo cấu trúc SBT (Scala Build Tool) tiêu chuẩn. Để biên dịch và đóng gói chương trình, tôi chạy lệnh sau trong thư mục gốc của dự án:

[redacted]

Lệnh này biên dịch toàn bộ mã nguồn Scala và tạo ra file JAR tại `target/scala-2.12/wordcount_2.12-1`
Sau khi có file JAR, tôi submit job lên Spark:

[redacted]

Spark sẽ đọc file `words.txt` trực tiếp từ HDFS thông qua đường dẫn `hdfs://localhost:9000/hcmus/231` đã được hardcode trong mã nguồn. Quá trình này sử dụng thư viện Hadoop client tích hợp sẵn trong Spark để giao tiếp với NameNode và DataNode. Với chế độ `local[*]`, Spark sử dụng tất cả CPU core của máy để thực thi job mà không cần cluster YARN riêng biệt.

TODO: Thêm ảnh chụp màn hình kết quả spark-submit

Hình 17: Chạy spark-submit WordCount thành công

2.4. Kết quả

Sau khi chạy xong, kết quả được lưu vào file `results/part-00000` với định dạng tab-separated. Nội dung file `results.txt` thu được:

[redacted]

TODO: Thêm ảnh chụp màn hình kết quả cat results.txt

Hình 18: Nội dung file results.txt sau khi chạy WordCount

2.5. Kiểm thử

Để xác minh tính đúng đắn của chương trình trước khi chạy trên toàn bộ `words.txt`, tôi tự kiểm thử với một tập dữ liệu nhỏ gồm 10 từ bao phủ đủ 8 chữ cái mục tiêu:

Dữ liệu kiểm thử (`test_input.txt`):

[redacted]

Kết quả tính tay (không phân biệt hoa thường):

- **c**: cat → 1
- **f**: fish, Fire → 2
- **h**: House, hello → 2
- **i**: Imagine → 1
- **m**: Mango → 1

- **s**: Sun → 1
- **t**: the → 1
- **u**: umbrella → 1

Kết quả mong đợi (TSV):

[redacted]

Kết quả thực tế từ chương trình: (TODO: điền kết quả sau khi chạy kiểm thử và so sánh với kết quả tính tay bên trên để xác nhận chương trình hoạt động đúng.)

PHẦN 3: Cài đặt Cụm Hadoop Fully-Distributed

3.1. Kiến trúc cụm vật lý

Cụm Hadoop fully-distributed của nhóm gồm một node Master và hai node Worker, tất cả được kết nối qua mạng LAN nội bộ. Tôi (MSSV 23120099, Trưởng nhóm Leader A) phụ trách node Master chạy Arch Linux. Các thành viên còn lại của nhóm phụ trách các node Worker. Trong chế độ fully-distributed, mỗi node đảm nhận vai trò chuyên biệt: Master chạy các tiến trình quản lý (NameNode, ResourceManager), còn Worker chạy các tiến trình lưu trữ và tính toán (DataNode, NodeManager).

Node	Vai trò	Địa chỉ IP
master (23120099)	NameNode / ResourceManager	TODO
worker1 (B)	DataNode / NodeManager	TODO
worker2 (C/D)	DataNode / NodeManager	TODO

3.2. Cấu hình mạng và SSH

Để các node trong cụm có thể giao tiếp với nhau bằng tên thay vì địa chỉ IP, tôi chỉnh sửa file `/etc/hosts` trên tất cả các node, thêm các dòng ánh xạ tên sang IP tương ứng. Ví dụ nội dung file `/etc/hosts` trên node Master:

```
# Cấu hình /etc/hosts cho cụm Hadoop fully-distributed
# TODO: thay TODO bằng IP thực tế của từng node
```

Tiếp theo, tôi cấu hình SSH không mật khẩu (passwordless SSH) từ Master đến tất cả Worker để các script `start-dfs.sh` và `start-yarn.sh` có thể tự động SSH vào từng Worker để khởi động daemon mà không cần nhập mật khẩu:

```
# Tạo cặp khóa SSH trên node Master
                      4096                      ""

# Sao chép public key sang từng Worker (thay username và IP thực tế)



# Kiểm tra kết nối SSH không mật khẩu
                      "hostname"
                      "hostname"
```

Nội dung file `$HADOOP_HOME/etc/hadoop/workers` cần được cập nhật để liệt kê tên hoặc IP của các Worker node:

```
```

Đối với `core-site.xml`, giá trị của `fs.defaultFS` cần được đổi từ `hdfs://localhost:9000` sang `hdfs://master:9000` để các Worker node có thể kết nối đến NameNode trên Master thông qua tên hostname:

```
<configuration>
  <property>
    <name>            </name>
    <value>                  </value>
  </property>
</configuration>
```

TODO: Thêm ảnh chụp màn hình

Hình 19: Kiểm tra kết nối SSH không mật khẩu từ Master đến Worker

3.3. Khởi động và kiểm tra YARN

Sau khi cấu hình xong, tôi khởi động toàn bộ cụm từ node Master bằng hai script quen thuộc. Khác với chế độ pseudo-distributed, lần này các daemon sẽ được khởi động trên nhiều máy vật lý thực sự:

```
# Khởi động từ node Master
                # Tự động SSH vào Worker để khởi động DataNode
                # Tự động SSH vào Worker để khởi động NodeManager
```

Sau khi khởi động, tôi kiểm tra trạng thái trên từng node bằng `jps`:

```
# Trên Master — kỳ vọng có: NameNode, ResourceManager, SecondaryNameNode



# Trên mỗi Worker — kỳ vọng có: DataNode, NodeManager
                      "jps"
                      "jps"
```

Giao diện WebUI của NameNode có thể truy cập tại `http://<master-ip>:9870` để xem trạng thái HDFS và danh sách DataNode. Giao diện YARN ResourceManager có thể truy cập tại `http://<master-ip>:8088` để theo dõi các job đang chạy và lịch sử thực thi.

TODO: Thêm ảnh chụp màn hình

Hình 20: WebUI NameNode hiển thị các DataNode trong cụm fully-distributed

TODO: Thêm ảnh chụp màn hình

Hình 21: WebUI YARN ResourceManager tại cổng 8088

PHẦN 4: Kết luận

Qua bài lab này, tôi đã trải qua toàn bộ chu trình cài đặt, cấu hình và vận hành một cụm Hadoop — từ chế độ pseudo-distributed đơn giản trên máy cá nhân cho đến mô hình fully-distributed nhiều node. Đây là nền tảng thực hành không thể thiếu để hiểu sâu hơn về kiến trúc của hệ sinh thái Big Data.

Những kiến thức thu được:

- **Kiến trúc HDFS**: tôi hiểu rõ vai trò phân công giữa NameNode (quản lý metadata và namespace) và DataNode (lưu trữ dữ liệu thực tế dạng block), cũng như cơ chế replication giúp đảm bảo tính chịu lỗi (fault tolerance). Nguyên tắc *write once, read many* của HDFS phù hợp với các workload phân tích dữ liệu lớn.
- **Hệ thống YARN**: tôi nắm được mô hình quản lý tài nguyên hai tầng của YARN — ResourceManager toàn cục điều phối tài nguyên, còn ApplicationMaster từng job tự quản lý lifecycle của các task. Kiến trúc này tách biệt quản lý tài nguyên khỏi quản lý job, giúp YARN hỗ trợ nhiều framework (MapReduce, Spark, Flink) trên cùng một cluster.
- **Mô hình MapReduce và Spark**: qua bài toán WordCount, tôi thực hành triển khai mô hình lập trình phân tán theo paradigm Map-Reduce. Spark với API RDD cho phép xử lý dữ liệu in-memory, nhanh hơn đáng kể so với MapReduce truyền thống khi xử lý các tác vụ lặp lại (iterative).
- **Quản lý phân quyền HDFS**: hệ thống phân quyền của HDFS tương tự Unix (owner, group, others) nhưng vận hành trong môi trường phân tán, đòi hỏi sự nhất quán về tên người dùng trên nhiều node.

Khó khăn và bài học kinh nghiệm:

Khó khăn lớn nhất tôi gặp phải là sự không tương thích giữa Cluster ID sau khi format lại NameNode và ID cũ còn lưu trong thư mục DataNode, khiến DataNode từ chối kết nối vào cluster. Bài học rút ra là: trong môi trường production, không bao giờ format lại NameNode trừ

khi xóa sạch toàn bộ dữ liệu DataNode đồng thời. Ngoài ra, việc sử dụng Arch Linux — hệ điều hành không phổ biến trong cộng đồng Hadoop — đôi khi gây ra các vấn đề về tên giao diện mạng không theo chuẩn truyền thống, đòi hỏi phải tìm hiểu thêm để cấu hình đúng.

Kinh nghiệm thực hành qua bài lab này là bước đệm quan trọng để tiếp cận các chủ đề nâng cao hơn trong môn học, bao gồm tối ưu hóa job Spark, tuning cấu hình YARN, và tích hợp Hadoop với các hệ thống lưu trữ và xử lý hiện đại như Hive, HBase, và Kafka.